Guía Técnica del Proyecto: Desarrollo de Aplicación Web Frontend

Objetivo General

Diseñar, maquetar, programar y desplegar una aplicación web del lado cliente (*Single Page Application* - SPA) viable y con diseño adaptativo, integrando semántica HTML5, maquetación CSS moderna, interactividad asíncrona con JavaScript nativo y arquitectura basada en componentes reactivos utilizando Vue.js.

Primera Entrega: Semántica Estructural, Maquetación Responsive y Versionamiento

Fecha límite de entrega: Viernes, 23 de octubre de 2026.

Propósito: Construir el esqueleto y la interfaz visual adaptativa del proyecto sin recurrir aún a interactividad compleja, asegurando accesibilidad, rendimiento y control de versiones.

Requerimientos y Entregables: Se entregará el enlace a un repositorio público en GitHub (con commits descriptivos del progreso) que contenga:

1. **Estructura Semántica (HTML5 Puro):**
    - Marcado estructurado sin abusar de contenedores genéricos (div). Empleo estricto de etiquetas semánticas: <header>, <nav>, <main>, <section>, <article>, <aside> y <footer>.
    - Formularios completos para la captura de datos del negocio con atributos de validación nativos (required, pattern, min, max, type="email", type="date").
    - Inclusión correcta de metadatos en el <head> (viewport, charset, title, descripción) e integración de recursos multimedia y gráficos (<picture>, <svg> o <img> optimizados).
2. **Maquetación y Diseño Adaptativo (CSS3):**
    - Estructura modular de estilos vinculada externamente (styles.css).
    - Implementación de sistemas de distribución espacial modernos: CSS Grid Layout para la macro-estructura de la página y Flexbox para componentes internos de navegación, tarjetas y barras de herramientas.

- Diseño Responsivo (*Mobile First* o *Desktop First*): Implementación de al menos tres *breakpoints* con Media Queries (móvil, tablet, escritorio) garantizando legibilidad y navegación fluida.

- Integración de recursos tipográficos e iconográficos externos vía CDN.

- *(Opcional/Complementario):* Uso justificado de utilidades o componentes de frameworks visuales (Bootstrap o Material Design) sin sobreescribir la maquetación personalizada exigida en Grid/Flexbox.

Avance 2: Lógica de Interfaz, Consumo de APIs y Persistencia Local

Fecha límite de entrega: Viernes, 13 de noviembre de 2026.

Propósito: Dotar a la aplicación de dinamismo y procesamiento en el cliente mediante JavaScript nativo, gestionando el flujo asíncrono de datos y almacenando información en el navegador.

Requerimientos y Entregables: Actualización del repositorio de GitHub con los siguientes módulos de JavaScript integrados:

1. **Manipulación del DOM y Gestión de Eventos:**

   - Captura e interceptación de eventos de usuario (submit, click, input, change) para actualizar vistas en tiempo real sin recargar la página (event.preventDefault()).

   - Creación, modificación y eliminación dinámica de elementos en el DOM en respuesta a las acciones del usuario.

2. **Consumo de Datos Asíncrono (Peticiones HTTP):**

   - Implementación de llamadas asíncronas utilizando la API nativa Fetch o la librería Axios mediante sintaxis moderna async / await.

   - Consumo de un API REST externa real (pública) o un servicio simulado (ej. MockAPI, JSONPlaceholder o un archivo data.json local estructurado).

   - Manejo de códigos de estado HTTP (200, 404, 500) y captura de errores mediante bloques try / catch, mostrando mensajes visuales de carga (*loaders*) y error al usuario.

3. **Persistencia del Lado Cliente (Browser API):**

   - Uso de localStorage para almacenar, actualizar y recuperar colecciones de datos en formato JSON. Los datos ingresados en los formularios deben persistir tras recargar el navegador.

4. **Validaciones Dinámicas de Negocio:**

    - Validación en tiempo real de campos de entrada con retroalimentación visual inmediata (resaltado de bordes, alertas informativas de error en línea) antes de procesar cualquier información.

Entrega Final: Implementación SPA con Vue.js y Despliegue en Producción

Fecha límite de entrega: Viernes, 27 de noviembre de 2026.

Propósito: Migrar y refactorizar la lógica hacia un entorno profesional de desarrollo basado en componentes reactivos con Vue.js, finalizando con la puesta en marcha de la aplicación en un servidor productivo.

Requerimientos y Entregables: Repositorio en GitHub con estructura de proyecto Vue.js (Vite o Vue CLI) y enlace funcional a la aplicación publicada en Internet:

1. **Arquitectura Basada en Componentes:**

    - Modularización de la interfaz en componentes de archivo único (*Single File Components* - .vue), separando vistas principales (views) y elementos reutilizables (components como tarjetas, modales, barras de navegación, tablas).

    - Paso de datos unidireccional y bidireccional mediante props y eventos personalizados ($emit o custom events).

2. **Enrutamiento de la Aplicación (Vue Router):**

    - Configuración de navegación SPA sin recarga de página mediante Vue Router.

    - Definición de al menos 3 rutas principales (ej. Inicio, Catálogo/Gestión, Formulario de Registro/Operación, Detalle de Ítem).

3. **Gestión de Estados y Formularios Reactivos:**

    - Uso de directivas clave de Vue (v-model, v-for, v-if / v-else, v-bind, v-on) para manejar la reactividad en formularios y listados dinámicos.

    - Manejo de estado centralizado o compartido para asegurar consistencia de los datos entre diferentes vistas.

4. **Despliegue en Ambiente de Producción:**

    - Generación del paquete de distribución optimizado (build de producción).

- Publicación y despliegue continuo de la aplicación en una plataforma de alojamiento en la nube para frontend (Vercel, Netlify o GitHub Pages).

5. **Documentación del Repositorio** (README.md)**:**

   - Descripción del proyecto, arquitectura de carpetas, instrucciones de instalación local (npm install, npm run dev) y enlace directo a la URL de producción.

6. **Sustentación Técnica:**

   - Demostración en vivo de la arquitectura de componentes y defensa individual del código fuente.